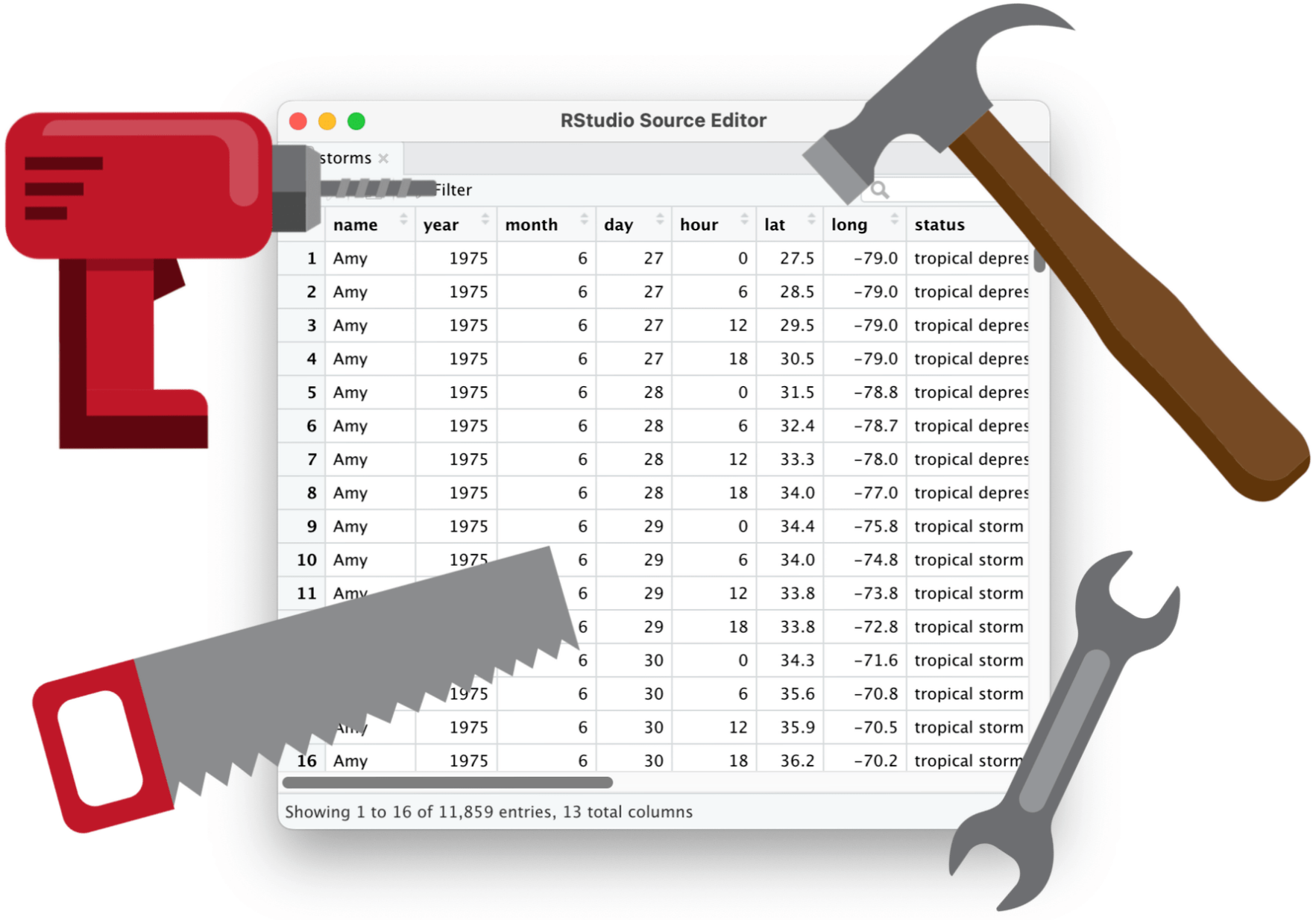
Data manipulation in R

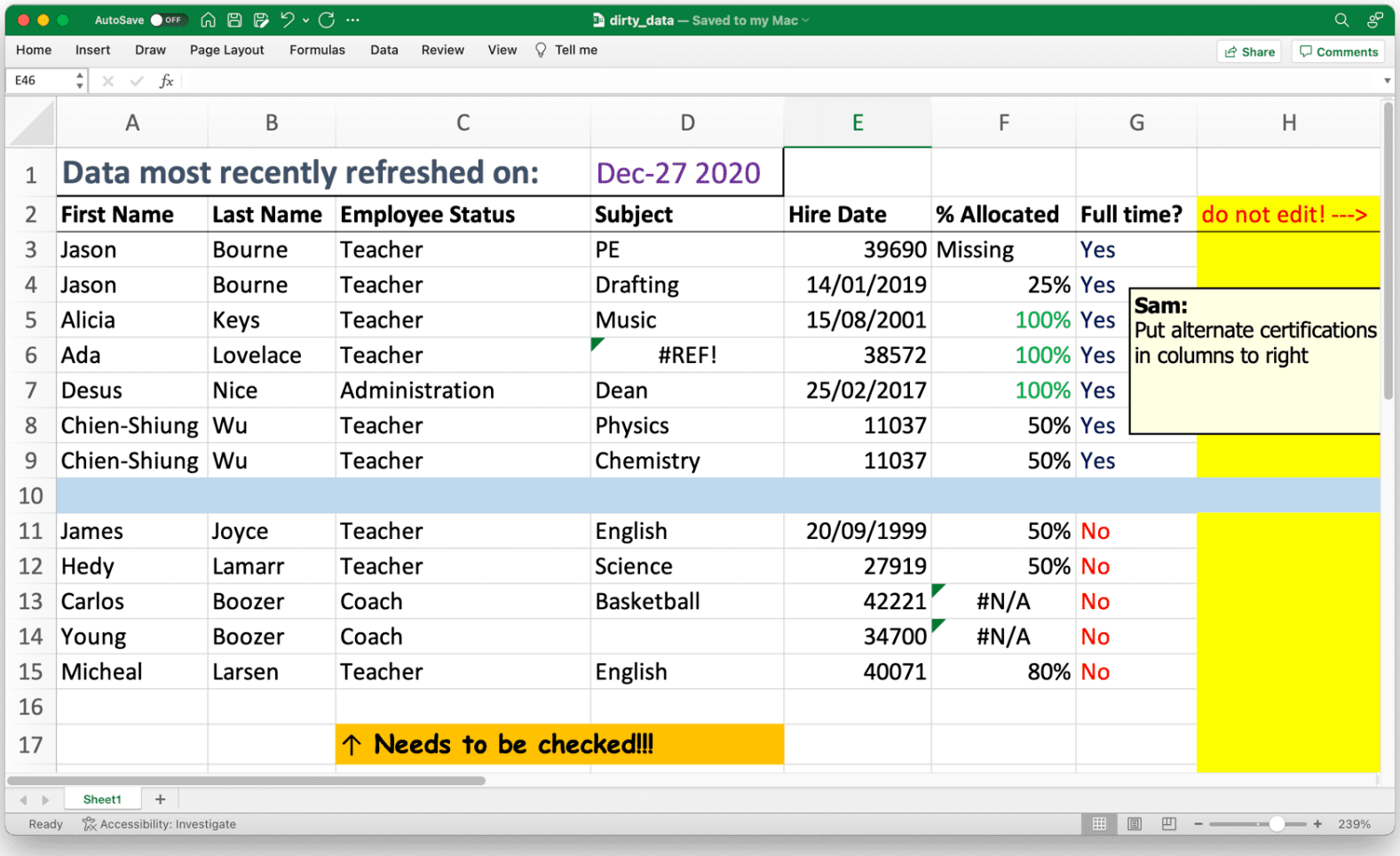
Part 1



	name	year	month	day	hour	lat	long	status
1	Amy	1975	6	27	0	27.5	-79.0	tropical depres
2	Amy	1975	6	27	6	28.5	-79.0	tropical depres
3	Amy	1975	6	27	12	29.5	-79.0	tropical depres
4	Amy	1975	6	27	18	30.5	-79.0	tropical depres
5	Amy	1975	6	28	0	31.5	-78.8	tropical depres
6	Amy	1975	6	28	6	32.4	-78.7	tropical depres
7	Amy	1975	6	28	12	33.3	-78.0	tropical depres
8	Amy	1975	6	28	18	34.0	-77.0	tropical depres
9	Amy	1975	6	29	0	34.4	-75.8	tropical storm
10	Amy	1975	6	29	6	34.0	-74.8	tropical storm
11	Amy		6	29	12	33.8	-73.8	tropical storm
			6	29	18	33.8	-72.8	tropical storm
			6	30	0	34.3	-71.6	tropical storm
		1975	6	30	6	35.6	-70.8	tropical storm
		1975	6	30	12	35.9	-70.5	tropical storm
16	Amy	1975	6	30	18	36.2	-70.2	tropical storm

Showing 1 to 16 of 11,859 entries, 13 total columns

You will spend most of your time cleaning data.



dirty_data — Saved to my Mac

Home Insert Draw Page Layout Formulas Data Review View Tell me Share Comments

E46

	A	B	C	D	E	F	G	H
1	**Data most recently refreshed on:**			Dec-27 2020				
2	**First Name**	**Last Name**	**Employee Status**	**Subject**	**Hire Date**	**% Allocated**	**Full time?**	do not edit! --->
3	Jason	Bourne	Teacher	PE	39690	Missing	Yes	
4	Jason	Bourne	Teacher	Drafting	14/01/2019	25%	Yes	
5	Alicia	Keys	Teacher	Music	15/08/2001	100%	Yes	
6	Ada	Lovelace	Teacher	#REF!	38572	100%	Yes	
7	Desus	Nice	Administration	Dean	25/02/2017	100%	Yes	
8	Chien-Shiung	Wu	Teacher	Physics	11037	50%	Yes	
9	Chien-Shiung	Wu	Teacher	Chemistry	11037	50%	Yes	
10								
11	James	Joyce	Teacher	English	20/09/1999	50%	No	
12	Hedy	Lamarr	Teacher	Science	27919	50%	No	
13	Carlos	Boozer	Coach	Basketball	42221	#N/A	No	
14	Young	Boozer	Coach		34700	#N/A	No	
15	Micheal	Larsen	Teacher	English	40071	80%	No	
16								
17			↑ Needs to be checked!!!					

Sam:
Put alternate certifications in columns to right

Sheet1

Ready Accessibility: Investigate 239%

Session overview

1. Alternative packages
2. Pipes
3. Essential data manipulation tasks

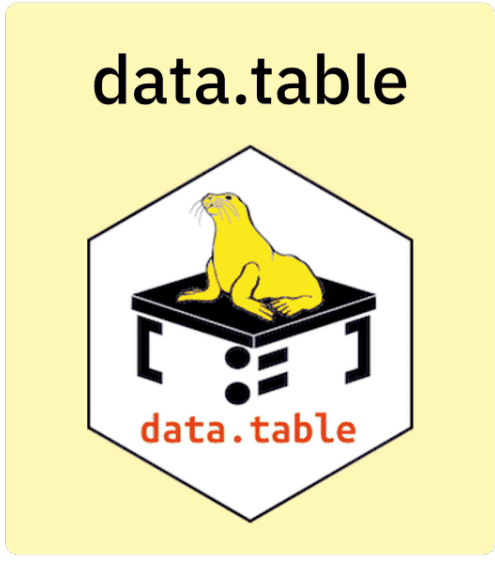
   a) Select columns or rows
   b) Sorting a dataset
   c) Creating or modifying columns
   d) Combining datasets (next session)
   e) Reshaping a dataset (next session)
   f) Grouping and summarising data (next session)

4. Recoding variables

There are many approaches to data manipulation in R.







- Use the right tool for the job.
- Use whatever feels most comfortable and productive.

What do I use?

The tidyverse

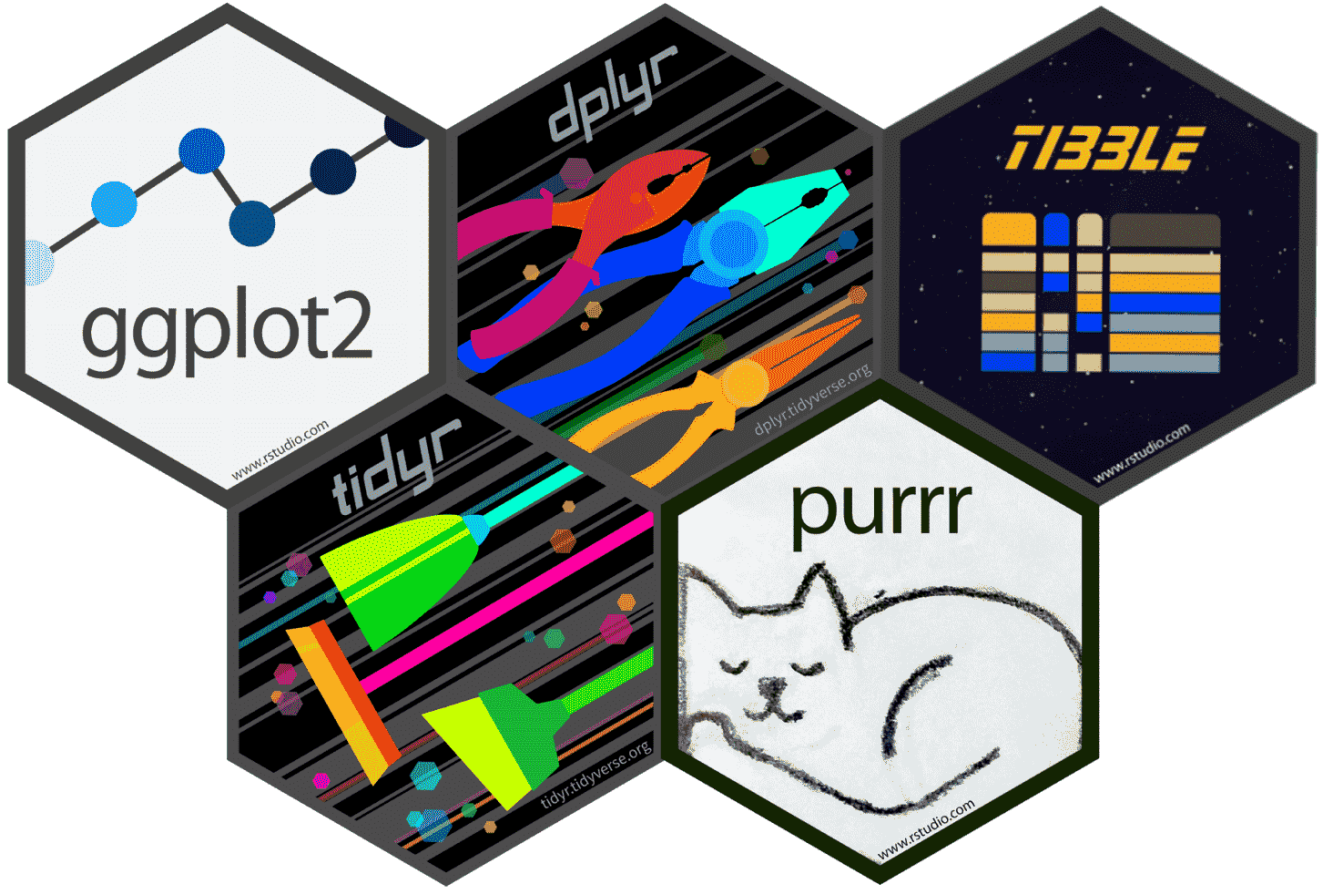
"an opinionated collection of R packages designed for data science. All packages share an underlying philosophy and common APIs".

www.tidyverse.org

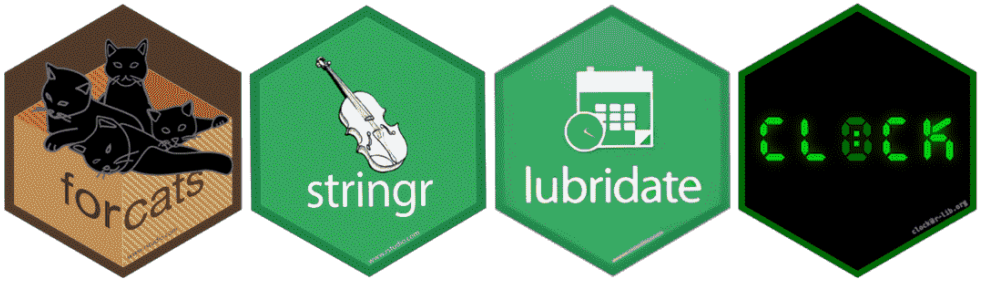




Core:



For specific data types:



For import/export:



You don’t need to learn all these packages.

Just use what you need.

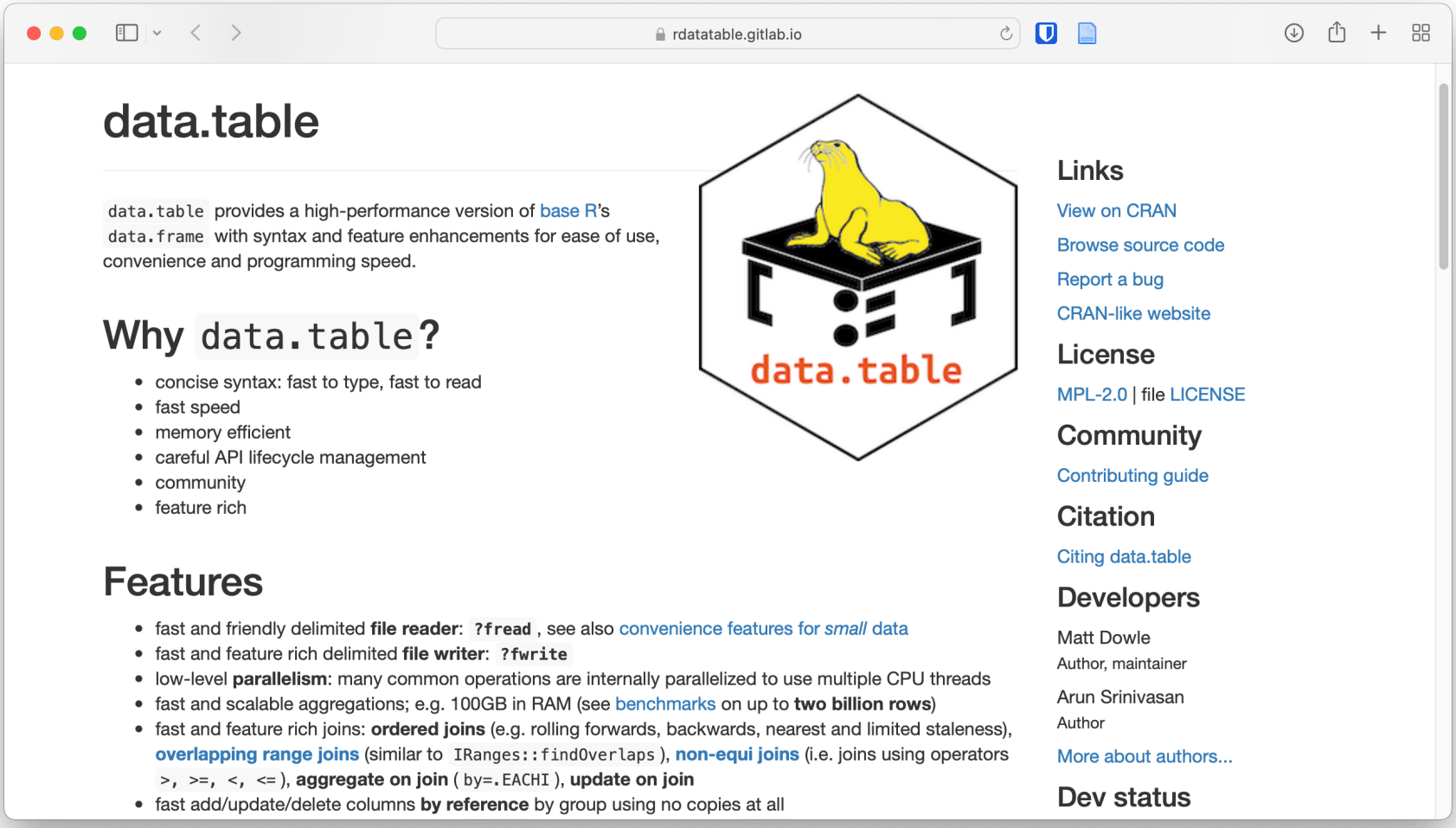
(Use the “Reference” help pages).

This session uses functions from `dplyr`.

I can never remember which functions come from which package.

It’s fine.

https://rdatatable.gitlab.io/data.table/



See this page for a comparison of `dplyr` and `data.table`

See also: dtplyr

Pipes

We can use |> to chain commands together

R code is traditionally written as a series of statements.

```
df <- haven::read_dta("stata_dataset.dta")
df <- df[df$age > 18, ]
df$log_income <- log(df$income)
df$female <- df$gender == "Female"
```

We can use |> to chain commands together

R code is traditionally written as a series of statements.

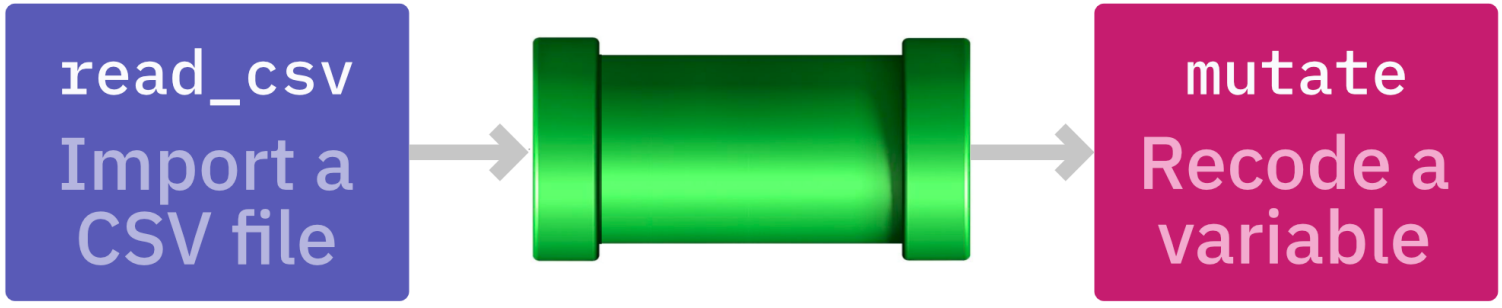
```
df <- haven::read_dta("stata_dataset.dta")
df <- df[df$age > 18, ]
df$log_income <- log(df$income)
df$female <- df$gender == "Female"
```

The pipe allows us to chain together several statements.

```
df <- haven::read_dta("stata_dataset.dta") |>
    filter(age > 18) |>
    mutate(log_income = log(income),
           female = gender == "Female")
```

How does it work?

A pipe takes output from one command and uses it as input to the next.



They are written as `|>` at the end of a line.

You need to save the output by assigning to an object.

```
clean_data <- messy_data |>
    mutate(...) |>
    pivot_wider(...)
```

Session overview

1. ~~Alternative packages~~
2. ~~Pipes~~
3. Essential data manipulation tasks
4. Recoding variables

Essential data manipulation tasks

a) Select columns or rows

b) Sorting a dataset

c) Creating or modifying columns

Let's load some data...

```
> library(tidyverse)
> data(starwars)      # Load a built-in dataset
> head(starwars)
# A tibble: 6 x 14
  name      height  mass hair_color skin_color eye_color birth_year
  <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl>
1 Luke Sk~    172    77 blond      fair       blue             19
2 C-3PO       167    75 <NA>       gold       yellow          112
3 R2-D2        96    32 <NA>       white, bl~ red              33
4 Darth V~    202   136 none       white      yellow           41.9
5 Leia Or~    150    49 brown      light      brown            19
6 Owen La~    178   120 brown, gr~ light      blue             52
# i 7 more variables: sex <chr>, gender <chr>, homeworld <chr>,
#   species <chr>, films <list>, vehicles <list>,
#   starships <list>
```

- Get a list of the variables in this data frame.
- How many rows and columns are there?

a) Selecting rows and columns

We've already seen how subsetting can be used to select parts of objects.

For example, we can select rows and columns by number:

```
starwars[1:5, ]
starwars[, c(1, 3, 8)]
```

Or by name:

```
starwars$height
starwars[, c("birth_year", "homeworld")]
```

Select and drop columns with `select`

```
starwars |>
    select(birth_year, homeworld)

# Negate to remove a column
starwars |>
    select(-eye_color)

# Select several columns
starwars |>
    select(starts_with("h"),
           ends_with("color"),
           matches("or$"))
```

You can rename at the same time.

```
starwars |>
    select(new_name = old_name)
```

To rename without dropping other variables, use `rename`:

```
starwars |>
    rename(new_name = old_name)
```

Select rows with `filter`

`filter` selects rows based on a condition.

For example, select all rows where `mass` is above 100:

```
starwars |>
    filter(mass > 100)
```

Separate multiple conditions with a comma:

```
starwars |>
    filter(mass > 100,
           eye_color == "yellow",
           homeworld == "Tatooine")
```

More conditions: %in%, | and !

```r
# Is the value one of a set?
starwars |>
    filter(eye_color %in% c("red", "blue"))

# OR: keep rows where either is TRUE
starwars |>
    filter(height > 200 | mass > 100)

# NOT: reverse a condition
starwars |>
    filter(!eye_color %in% c("red", "blue"))
```

b) Sorting a dataset with `arrange`

```
starwars |>
    arrange(height)

# Sorting on multiple columns
starwars |>
    arrange(height, mass)

# Sort in descending order
starwars |>
    arrange(desc(height))
```

Practical: pipes, select, filter, and arrange

From the `starwars` data frame:

1. Select the columns `height`, `mass`, `gender`, and `species`.
2. Filter to select rows with `height` less than 191 and with `species` equal to "Human".
3. Sort the result by `height`.

Use pipes to combine each operation; store the result as a new data frame.

Practical: pipes, select, filter, and arrange

```
humans <- starwars |>
  select(height, mass, gender, species) |>
  filter(height < 191, species == "Human") |>
  arrange(height)
```

humans has 26 rows, from the shortest (150cm) upwards. Use `desc(height)` to sort from tallest to shortest.

c) Create or modify variables with `mutate`

In base R, we can create new columns using the assignment operator:

```
df$eligible <- TRUE
df$log_income <- log(df$income)
df$female <- df$sex == "Female"
```

We can transform existing variables using subsetting:

Replace "Not applicable" with **NA**.

```
df$wstat[df$wstat == "Not applicable"] <- NA
```

Create binary measure of age:

```
df$older <- 0
df$older[df$age > 50] <- 1
```

But `mutate` makes this easier.

```
df <- df |>
    mutate(eligible = TRUE,
           log_income = log(income),
           female = sex == "Female")
```

But `mutate` makes this easier.

```
df <- df |>
    mutate(eligible = TRUE,
           log_income = log(income),
           female = sex == "Female")
```

Things to note:

- We're not using subsetting or quoting.
- We can include multiple statements inside a single `mutate` function.
- We can use earlier computations in later ones.
- We need to store the result.

Practical: Creating and modifying columns

1. Load the `tidyverse` package and the `mtcars` dataset.
2. Add a new column indicating whether a car weighs over 3000 lbs (i.e. `wt > 3`).
    i. Using subsetting
    ii. Using mutate
3. Tabulate this new column against the number of cylinders (`cyl`).

Practical: Creating and modifying columns

2. i. Using subsetting

```
mtcars$heavy <- mtcars$wt > 3
```

ii. Using mutate

```
mtcars <- mtcars |>
   mutate(heavy = wt > 3)
```

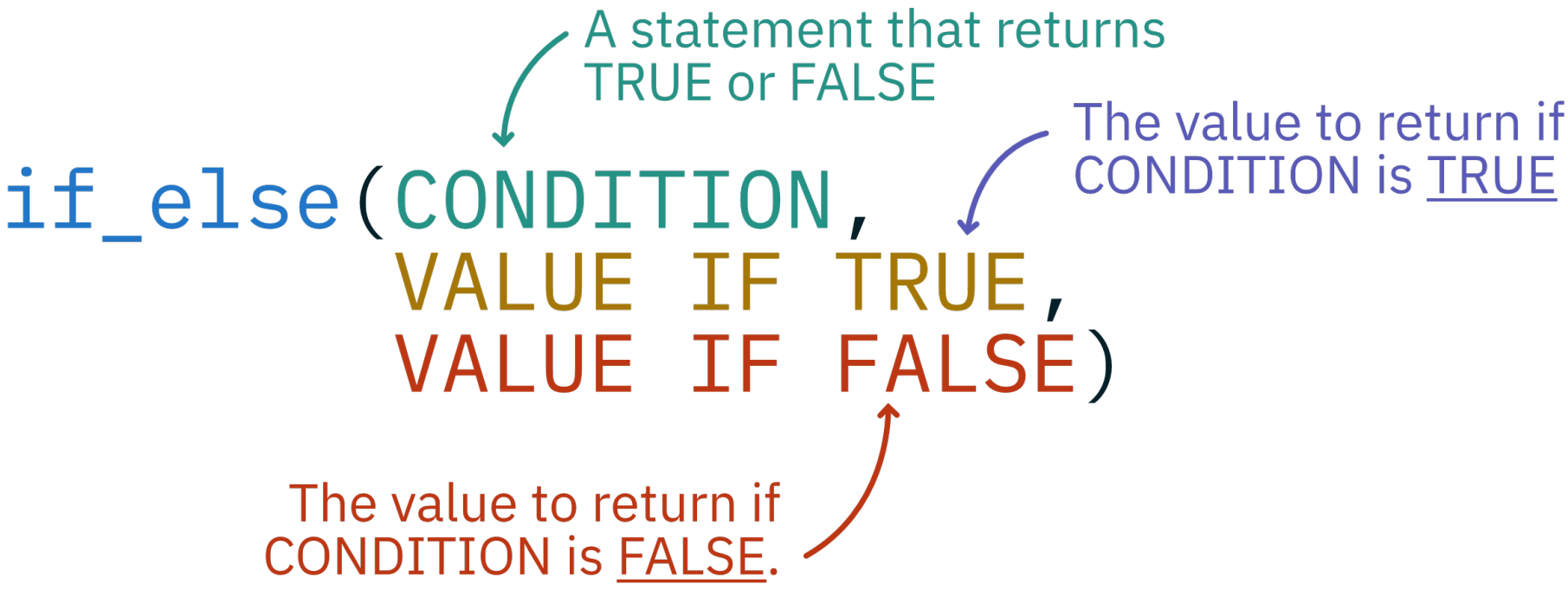
3. Tabulate against `cyl`

```
table(mtcars$heavy, mtcars$cyl)
```

Recoding variables

Recoding with `if_else` and `case_when`

Two functions that solve many common data cleaning tasks.



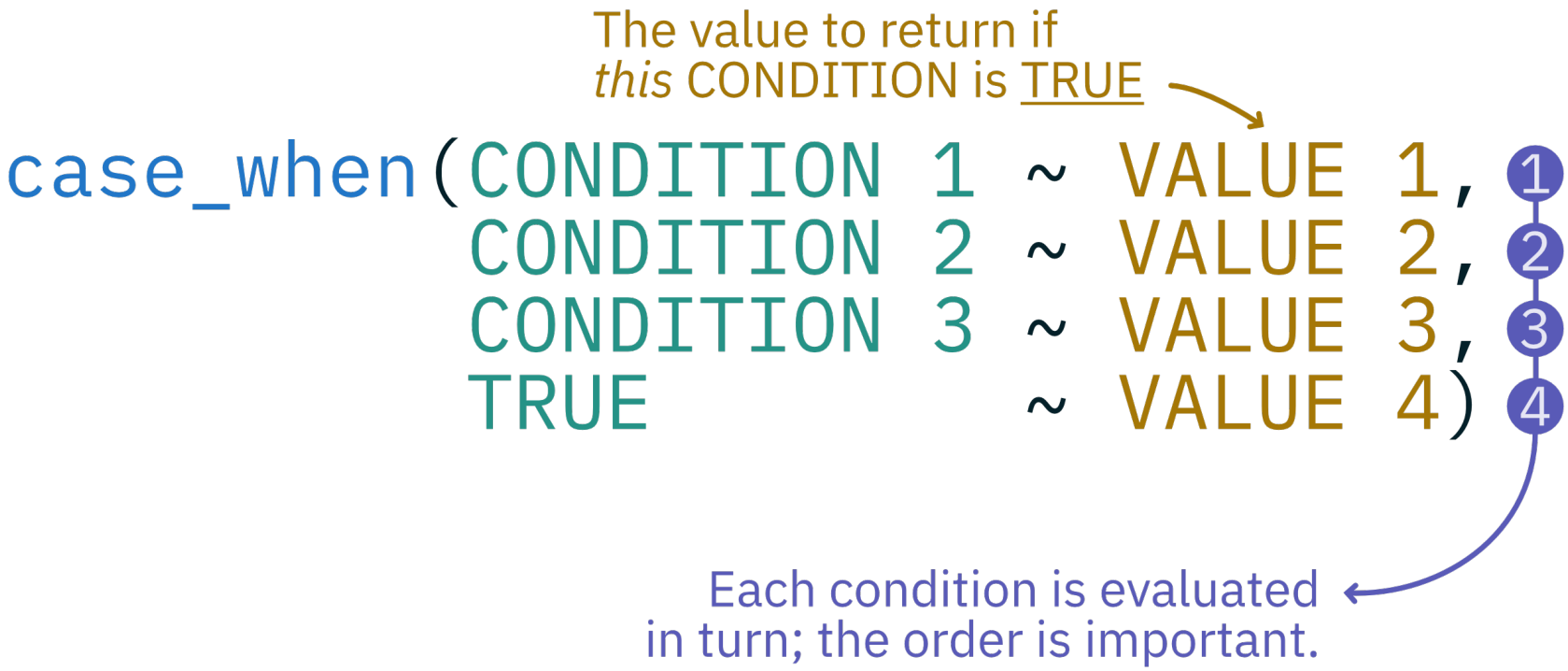
```
> starwars |>
+   mutate(weight = if_else(mass > 80,
+                           "Heavy",
+                           "Not heavy")) |>
+   select(mass, weight)
# A tibble: 87 x 2
   mass weight
  <dbl> <chr>
1    77 Not heavy
2    75 Not heavy
3    32 Not heavy
4   136 Heavy
5    49 Not heavy
6   120 Heavy
# i 81 more rows
```

However, if you're creating a binary indicator (`TRUE`/`FALSE`), there's a simpler option:

```
> starwars |>
+   mutate(heavy = mass > 80) |>
+   select(mass, heavy)
# A tibble: 87 x 2
   mass heavy
  <dbl> <lgl>
1    77 FALSE
2    75 FALSE
3    32 FALSE
4   136 TRUE
5    49 FALSE
6   120 TRUE
# i 81 more rows
```

case_when

To evaluate multiple conditions in order.



```
> starwars |>
+   mutate(size = case_when(
+     height < 100  ~ "Short",
+     height < 180  ~ "Medium",
+     height >= 180 ~ "Tall",
+     .default = "Unknown")) |>
+   select(height, size)
# A tibble: 87 x 2
  height size
   <int> <chr>
1    172 Medium
2    167 Medium
3     96 Short
4    202 Tall
5    150 Medium
6    178 Medium
# i 81 more rows
```

R2-D2 (`height` = 96) meets *both* `height < 100` and `height < 180`. The first matching condition wins, so R2-D2 is `"Short"`.

```
> starwars |>
+   mutate(size = case_when(...)) |>
+   count(size)
# A tibble: 4 x 2
  size         n
  <chr>    <int>
1 Medium      30
2 Short        7
3 Tall        44
4 Unknown      6
```

Six characters have a missing `height`. They match none of the conditions, so get the `.default` value, `"Unknown"`.
(`TRUE ~ "Unknown"` does the same thing.)

`recode_values()` swaps values one-for-one (`old ~ new`). Note `default`, not `.default`.

```
> starwars |>
+   mutate(sex2 = recode_values(
+     sex,
+     "male"   ~ "Male",
+     "female" ~ "Female",
+     default  = "Other")) |>
+   count(sex, sex2)
# A tibble: 5 x 3
  sex           sex2       n
  <chr>         <chr>  <int>
1 female        Female    16
2 hermaphroditic Other     1
3 male          Male      60
4 none          Other      6
5 <NA>          Other      4
```

Phew, that was a lot...

1. Alternative packages
2. Pipes
3. Essential data manipulation tasks

a) Select columns or rows
b) Sorting a dataset
c) Creating or modifying columns
d) Combining datasets (next session)
e) Reshaping a dataset (next session)
f) Grouping and summarising data (next session)

4. Recoding variables with `if_else`, `case_when`, and `recode_values`